

Enseignant : BELACEL Madani

Module : Informatique

Niveau : ENS — 1ère année — Département de français

Durée : 1h30

Année universitaire : 2025-2026

Objectifs pédagogiques spécifiques

À l'issue de cette séance, l'étudiant est capable de :

- Définir une structure de données et expliquer son rôle algorithmique
- Distinguer accès direct ($O(1)$) et accès séquentiel ($O(n)$) dans un tableau
- Implémenter les opérations push, pop, peek d'une pile (LIFO) et enqueue, dequeue d'une file (FIFO)
- Modéliser une liste chaînée avec nœuds et pointeurs
- Choisir la structure adaptée (tableau, pile, file, liste chaînée) selon le problème

Plan du cours

1. Notion de structure de données — 5 min
2. Tableaux (array) — 15 min
3. Piles LIFO — 20 min
4. Files FIFO — 20 min
5. Listes chaînées — 15 min
6. Comparaison tableau vs liste chaînée — 10 min
7. QCM et conclusion — 5 min

1. Notion de structure de données (5 min)

Une structure de données est une organisation logique des données en mémoire permettant des opérations efficaces (ajout, suppression, recherche, parcours).

Critères de choix : type d'accès (direct/séquentiel), dynamisme (taille fixe/variable), type d'opérations fréquentes.

2. Tableaux (array) (15 min)

Un tableau est une zone mémoire contiguë contenant des éléments de même type, chacun repéré par un indice.

```
// Pseudo-code : déclaration et accès
notes: TABLEAU[10] D'ENTIERS
notes[0] ← 15
notes[1] ← 12
notes[2] ← 18
```

```
// Parcours et somme
somme ← 0
POUR i DE 0 À 9 FAIRE
  somme ← somme + notes[i]
FIN POUR
```

Complexités fondamentales :

- Accès par indice : $O(1)$ — calcul direct de l'adresse mémoire : $\text{addr} = \text{base} + i \times \text{taille_élément}$
- Parcours séquentiel : $O(n)$ — il faut visiter chaque élément
- Insertion au milieu : $O(n)$ — décalage des éléments suivants

Propriétés : taille fixe (statique), contiguë en mémoire, accès aléatoire instantané.

Exemple pédagogique NSI : un tableau de 30 notes pour une classe. Calcul de la moyenne en $O(n)$. Recherche du maximum en $O(n)$.

```
graph LR
  subgraph "Tableau notes[5] en mémoire"
    A["Indice 0<br/>15"]
    B["Indice 1<br/>12"]
    C["Indice 2<br/>18"]
    D["Indice 3<br/>10"]
    E["Indice 4<br/>14"]
  end
  ACCES["Accès notes[2] → 0(1)<br/>Adresse = base + 2×4 octets"] --> C

  style A fill:#3498db,color:#fff
  style B fill:#3498db,color:#fff
  style C fill:#e74c3c,color:#fff
  style D fill:#3498db,color:#fff
  style E fill:#3498db,color:#fff
  style ACCES fill:#2ecc71,color:#fff
```

3. Piles — LIFO (20 min)

Principe : Last In, First Out — le dernier élément ajouté est le premier retiré.

Analogie : une pile d'assiettes. On pose la nouvelle assiette sur le dessus ; on prend toujours celle du dessus.

```
// Opérations sur une pile
PILE pile ← VIDE

empiler(pile, 10) // [10]
empiler(pile, 20) // [10, 20]
empiler(pile, 30) // [10, 20, 30]
x ← depiler(pile) // x = 30, pile = [10, 20]
y ← depiler(pile) // y = 20, pile = [10]
```

Opération	Rôle	Complexité
<code>push(v)</code>	Ajouter au sommet	$O(1)$
<code>pop()</code>	Retirer et renvoyer le sommet	$O(1)$
<code>peek()</code>	Consulter le sommet sans retirer	$O(1)$
<code>is_empty()</code>	Vérifier si la pile est vide	$O(1)$

```

graph TB
subgraph "Pile LIFO – État après push(10), push(20), push(30)"
S3["30 ← SOMMET (push / pop)"]
S2["20"]
S1["10 ← BASE"]
end

subgraph "Opérations"
P1["push(40) → [10, 20, 30, 40]"]
P2["pop() → retourne 30 → [10, 20]"]
P3["peek() → retourne 30 (sans retirer)"]
end

S3 --> P1
S3 --> P2
S3 --> P3

style S3 fill:#e74c3c,color:#fff
style S2 fill:#f39c12,color:#fff
style S1 fill:#3498db,color:#fff
style P1 fill:#2ecc71,color:#fff
style P2 fill:#2ecc71,color:#fff
style P3 fill:#2ecc71,color:#fff

```

Exemples concrets :

- Ctrl+Z (annulation) : chaque action est empilée, Ctrl+Z dépile
- Historique navigateur : bouton « Précédent » dépile les URLs visitées
- Pile d'appels (call stack) : le système empile les appels de fonctions

Enseignement NSI : la pile d'appels explique la récursivité. Chaque appel récursif empile un nouveau cadre ; le retour dépile.

4. Files — FIFO (20 min)

Principe : First In, First Out — le premier élément ajouté est le premier retiré.

Analogie : une file d'attente à la cantine. Le premier arrivé est le premier servi.

```

// Opérations sur une file
FILE file ← VIDE

enfiler(file, 10) // [10]
enfiler(file, 20) // [10, 20]
enfiler(file, 30) // [10, 20, 30]
x ← defiler(file) // x = 10, file = [20, 30]
y ← defiler(file) // y = 20, file = [30]

```

Opération	Rôle	Complexité
<code>enqueue(v)</code>	Ajouter à la fin (queue)	O(1)
<code>dequeue()</code>	Retirer et renvoyer le début (tête)	O(1)
<code>front()</code>	Consulter la tête sans retirer	O(1)
<code>is_empty()</code>	Vérifier si la file est vide	O(1)

```

graph LR
subgraph "File FIFO"
direction LR
TETE["TÊTE<br/>(dequeue)"]
end

```

```

F1["10"]
F2["20"]
F3["30"]
QUEUE["QUEUE<br/>(enqueue)"]
end

TETE --> F1
F1 --> F2
F2 --> F3
F3 --> QUEUE

ENTREE["enqueue(40) → [10, 20, 30, 40]"] --> QUEUE
TETE --> SORTIE["dequeue() → 10 → [20, 30]"]

style F1 fill:#e74c3c,color:#fff
style F2 fill:#f39c12,color:#fff
style F3 fill:#3498db,color:#fff
style TETE fill:#2ecc71,color:#fff
style QUEUE fill:#2ecc71,color:#fff
style ENTREE fill:#9b59b6,color:#fff
style SORTIE fill:#9b59b6,color:#fff

```

Exemples concrets :

- File d'impression : les documents s'impriment dans l'ordre d'envoi
- Tampon clavier : les touches sont traitées dans l'ordre de frappe
- Gestionnaire de tâches : files d'attente des processus CPU

Enseignement SNT : simuler une file d'attente avec les élèves (jeu de rôle). Chaque élève reçoit un ticket numéroté.

5. Listes chaînées (15 min)

Une liste chaînée est une structure dynamique où chaque élément (nœud) contient une valeur et un pointeur vers le nœud suivant.

```

// Structure d'un nœud
Noeud {
  valeur: ENTIER
  suivant: POINTEUR_VERS_Noeud
}

graph LR
  subgraph "Liste chaînée simple"
    N1["Nœud 1<br/>valeur: 10<br/>suivant: *"]
    N2["Nœud 2<br/>valeur: 20<br/>suivant: *"]
    N3["Nœud 3<br/>valeur: 30<br/>suivant: null"]
  end

  TETE["TÊTE"] --> N1
  N1 -->|"pointeur"| N2
  N2 -->|"pointeur"| N3
  N3 --> FIN["null (fin)"]

  style N1 fill:#3498db,color:#fff
  style N2 fill:#2ecc71,color:#fff
  style N3 fill:#f39c12,color:#fff
  style TETE fill:#e74c3c,color:#fff
  style FIN fill:#95a5a6,color:#fff

```

Caractéristiques :

- Taille dynamique : pas de limite fixe, la mémoire est allouée au fur et à mesure
- Insertion au début : $O(1)$ — il suffit de modifier le pointeur de tête
- Insertion au milieu : $O(1)$ — si on a le pointeur vers le nœud précédent
- Accès par indice : $O(n)$ — obligé de parcourir depuis la tête
- Recherche : $O(n)$ — parcours séquentiel obligatoire

Pseudo-code insertion au début :

```
// Insérer valeur au début d'une liste chaînée
FONCTION inserer_debut(tete, valeur)
nouveau ← NOUVEAU_Noed(valeur)
nouveau.suivant ← tete
RETOURNER nouveau // nouvelle tête
FIN FONCTION
```

6. Comparaison et choix (10 min)

```
graph TB
    subgraph "Comparaison Tableau vs Liste chaînée"
        subgraph "Tableau"
            T1["✓ Accès  $O(1)$   
✓ Mémoire contiguë  
✗ Insertion au début  $O(n)$   
✗ Taille fixe"]
        end
        subgraph "Liste chaînée"
            L1["✓ Insertion  $O(1)$   
✓ Taille dynamique  
✗ Accès  $O(n)$   
✗ Mémoire dispersée"]
        end
    end

    style T1 fill:#3498db,color:#fff
    style L1 fill:#2ecc71,color:#fff
```

Critère	Tableau	Liste chaînée
Accès par indice	$O(1)$ — direct	$O(n)$ — séquentiel
Insertion début	$O(n)$ — décalage	$O(1)$ — pointeur
Insertion fin	$O(1)$ — si place	$O(1)$ — si queue connue
Suppression	$O(n)$ — décalage	$O(1)$ — pointeur
Recherche	$O(n)$	$O(n)$
Mémoire	Fixe, contiguë	Dynamique, dispersée
Cache CPU	Favorable (localité)	Défavorable

Guide de choix :

- Accès fréquent par position → Tableau (ex: liste de notes indicées)
- Insertions/suppressions fréquentes au début ou au milieu → Liste chaînée
- Opérations LIFO → Pile (ex: historique, annulation)
- Opérations FIFO → File (ex: file d'attente, tampon)
- Taille inconnue à l'avance → Liste chaînée

QCM d'auto-évaluation

1. Quelle est la complexité d'accès à un élément dans un tableau par son indice ?
a) $O(n)$ b) $O(1)$ ✓ c) $O(n^2)$
2. Dans une pile LIFO, quel élément retire-t-on avec `pop()` ?
a) Le premier ajouté b) Le dernier ajouté ✓ c) L'élément au milieu
3. Quelle analogie représente le mieux une file FIFO ?
a) Une pile d'assiettes b) Une file d'attente au guichet ✓ c) Un tableau d'affichage
4. Dans une liste chaînée, quel est le coût d'une insertion au début ?
a) $O(1)$ ✓ b) $O(n)$ c) $O(n^2)$
5. Quelle structure de données utilise le bouton « Précédent » du navigateur ?
a) File b) Pile ✓ c) Liste chaînée

Conclusion (5 min)

Les structures de données sont des outils algorithmiques essentiels. Un tableau favorise l'accès rapide ($O(1)$). Une liste chaînée favorise l'insertion dynamique ($O(1)$). La pile organise le « dernier arrivé, premier servi » (LIFO) ; la file organise le « premier arrivé, premier servi » (FIFO). Le choix de la structure détermine l'efficacité du programme.

Prochain cours : Réseaux et Internet — comprendre comment les ordinateurs communiquent.

Note pédagogique ENS

Transposition pour le secondaire :

- SNT Seconde : utiliser des activités débranchées pour piles (assiettes, livres) et files (file d'attente physique)
- NSI Première : implémenter une pile en Python avec une liste ; visualiser la pile d'appels récursifs
- Lien interdisciplinaire : en mathématiques, la pile sert à évaluer les expressions arithmétiques (notation polonaise inverse)

Conseil : faire tracer l'évolution de la pile et de la file au tableau étape par étape. Chaque élève manipule des post-its représentant les nœuds.

Bibliographie

- Cormen, Leiserson, Rivest & Stein — Introduction to Algorithms (ch. 10 : Elementary Data Structures)
- Aho, Ullman — Data Structures and Algorithms
- Downey — Think Python (ch. 17 : Stacks and Queues)
- Documentation Python : module `queue` (implémentations thread-safe)